KIRA AI Incident Report

Cluster Summary

The cluster is a single-node `minikube` environment running a microservice-oriented architecture (resembling an e-commerce or online boutique deployment). There are currently 7 deployments active, all scaled to 1 replica. While all workloads are currently reporting a status of `Running` and deployments show `1/1` available replicas, a highly correlated event has occurred: **6 out of the 8 running pods have restarted exactly once**.

Despite the active "SEV-2" alert flags indicating high restart counts, the services have recovered to a stable state, meaning there is no active, ongoing service outage, though transient disruption occurred.

Key Findings

1. **Synchronized Restart Pattern**: The pods `auth`, `boutique-postgres-0`, `frontend`, `order-service`, `product-service`, and `user-service` have all restarted exactly `1` time. The only exceptions are `gateway` and `orders`.
2. **Single-Node Failure Domain**: Every single pod is running on the `minikube` node.
3. **Abnormally Low Frontend CPU Usage**: The `frontend` pod shows an extremely low CPU utilization metric (`0.000013` cores or ~0.01m), indicating it is practically idle and may not be receiving or processing ingress traffic post-restart.
4. **Empty Log State**: Loki telemetry contains no logs, implying the restarts may have occurred prior to the current log buffer window, or logging output was lost during the container restarts.

Severity Assessment

Severity: SEV-3 (Moderate / Recovered)

* **Why?** All deployments are currently meeting their desired replica state (`1/1` available), and all pods are in the `Running` state. This is not an active SEV-1 (critical outage) or SEV-2 (degraded state with active user-facing impact). However, a synchronized restart across 75% of the cluster workloads indicates a node-level or control-plane anomaly that must be investigated to prevent future regressions.

Root Cause Analysis

Because the restarts occurred across both stateful (`boutique-postgres-0`) and stateless (`auth`, `frontend`, etc.) workloads simultaneously on a single-node cluster (`minikube`), we can rule out individual application code panics.

The three highly probable root causes are:

1. **Node Memory Pressure / OOM Killer**: The host system (or Minikube VM) ran out of physical memory. The Linux kernel Out-Of-Memory (OOM) killer or the Kubelet evicted/terminated multiple container runtimes to reclaim memory.
2. **Kubelet or Container Runtime Restart**: A restart of the host's `kubelet` daemon or container runtime (`docker`/`containerd`) would cause all running containers to be stopped and restarted simultaneously.
3. **Transient Node Reboot / Resource Starvation**: Since `minikube` runs locally, a host sleep cycle, VM pause, or temporary resource starvation may have caused the Kubelet to lose contact with the API server, subsequently restarting containers once the runtime state stabilized.

Recommended Remediation

1. **Investigate Node Events**: Run diagnostics on the `minikube` node to confirm if OOM killer events or Kubelet restarts occurred.
2. **Retrieve Prior Container Logs**: Examine the terminated container logs (`--previous` flag) for the restarted pods to locate exit codes (e.g., `Exit Code 137` indicates an OOM Kill).
3. **Analyze Resource Specifications**: Check if the deployments have resource requests and limits set. If limits are missing or too tight, the pods are highly vulnerable to OOM kills.

Preventative Measures

* **Set Resource Requests & Limits**: Implement strict CPU and Memory `requests` and `limits` for all deployments to allow the Kubelet to schedule workloads predictably and avoid node-level OOM cascades.
* **Proactive Resource Monitoring**: Set up Prometheus alerts for node memory utilization exceeding 80% (`node_memory_Active_bytes`).
* **Multi-Node Topologies (Production)**: For actual production environments, move away from single-node configurations to multi-AZ node groups to ensure high availability during node-level failures.

Exact kubectl Commands to diagnose and fix the issue

1. Diagnose Node Health & System Events
Inspect the node for recent memory/disk pressure and check cluster-wide events:
```bash
# Check node resources and general status
kubectl describe node minikube

# Sort cluster events by timestamp to see the sequence of restarts
kubectl get events -A --sort-by='.metadata.creationTimestamp'

# Check resource usage of the node and pods
kubectl top nodes
kubectl top pods
```

2. Check Previous Container Exit States
Identify if the restarted pods exited due to OOM (Exit Code 137) or system panic:
```bash
# Check the termination reason and exit code for a restarted pod
kubectl get pod boutique-postgres-0 -o jsonpath='{.status.containerStatuses[*].lastState}'

# Read logs of the previous container instance before it restarted
kubectl logs boutique-postgres-0 --previous --tail=100
kubectl logs frontend-75bc59b865-td9rw --previous --tail=100
kubectl logs user-service-76f9885d9b-q9dqf --previous --tail=100
```

3. Check for Missing Resource Limits
Verify if resource parameters are configured:
```bash
kubectl get deployments -o custom-columns=NAME:.metadata.name,CPU_LIMIT:.spec.template.spec.containers[*].resources.limits.cpu,MEM_LIMIT:.spec.template.spec.containers[*].resources.limits.memory
```